JIMBO

How this website works

The concept, the architecture, and how to change the styling

21 August 2026

An explanation of what has been built, why it is put together this way, and how to change the way it looks.

This is written for someone who does not build websites. There is no code to learn here — the aim is that you understand the shape of the thing well enough to make decisions about it, and to brief someone else if you ever need to.

Contents

1. The short version
2. What a website actually is
3. The problem with writing plain HTML
4. What Astro does about it
5. Build time, not visit time
6. The three layers
7. A page being built, step by step
8. What this buys you
9. Why not WordPress, Wix or Squarespace
10. Why not React or Next.js
11. Being fair: when plain HTML would have been fine
12. Changing how the site looks
13. The limits of this setup

---

1. The short version

Your website is **a folder of ordinary files** — HTML pages, images, videos, a stylesheet. Any web host on earth can serve it. There is no database, no login, no software running on a server that could break or need patching.

But writing those files by hand would be miserable, because a site of fourteen pages contains the same header, footer and menu fourteen times over.

So you do not write them by hand. You write **the content** (one file per project, one file per page) and **the templates** (what a project page looks like), and a tool called **Astro** combines the two and produces the fourteen finished pages for you.

You get the simplicity of plain files to host, without the tedium of maintaining them by hand. That is the entire idea.

---

2. What a website actually is

When someone visits your site, their browser asks your web host for a page, and the host sends back a file of HTML — text with markers around it saying “this is a heading”, “this is a paragraph”, “put this image here”.

Alongside it comes a **stylesheet** (CSS), which says what those things should look like: the colours, the fonts, the spacing, and how the layout should rearrange itself on a phone.

That is genuinely all a web page is. Everything else is a means of producing those files.

Broadly, there are two ways to produce them.

Ahead of time. The pages are made once, in advance, and sit on the host as finished files. When a visitor arrives, the host hands over a file it already had. This is what your site does.

On demand. Nothing exists in advance. When a visitor arrives, software on the server wakes up, queries a database, assembles the page on the spot, and sends it. This is what WordPress does.

Almost every practical difference between the options comes back to this one distinction.

---

3. The problem with writing plain HTML

Suppose you had written your fourteen pages by hand. Everything works. Then one of these entirely ordinary things happens.

Your phone number changes. It appears in the footer of every page. You open fourteen files, find it in each, change it, save. Miss one, and there is a dead number on your Contact page for a year.

You add a new project. You copy an existing project page and edit it. Then you add a card for it on the Work page. Then on the home page. Then you update the “previous / next” links on the two neighbouring projects. Five files edited to add one project, and four of those edits are ones you can silently forget.

You add a page called FAQ. It needs to appear in the top menu — which lives in all fourteen files. Fifteen now.

You decide the orange is too bright. It appears in buttons, links, headings and borders across every page.

None of this is difficult. It is just repetitive, and repetitive work performed by a human is work that eventually gets done inconsistently. The usual result is a site that slowly rots: the footer says 2019 on three pages, one project is missing from the Work listing, and the menu is different on the About page.

The problem is not that HTML is hard. The problem is duplication. The same header exists in fourteen places, so there are fourteen chances for it to drift out of step.

---

4. What Astro does about it

Astro removes the duplication by keeping each thing in exactly one place.

The header exists once, in a file called `Header.astro`. The look of a project page exists once. Your contact details exist once. Each project exists once, as a small file containing only what makes that project different from the others.

Then you run one command — `npm run build` — and Astro reads all of it, combines the pieces, and writes out the fourteen finished HTML pages.

The header still ends up in all fourteen pages. But **you** only ever maintain one copy, and the machine does the copying — which it does identically, every time, without getting bored.

Look at what the four scenarios above become:

THE CHANGE	BY HAND	WITH ASTRO
Phone number changes	Edit 14 files	Edit 1 line in `site.config.ts`
Add a project	Edit 5 files, remember all 5	Add 1 file
Add an FAQ page	Edit 15 files	Add 1 file with `nav: true`
Change the orange	Hunt across every page	Edit 1 line in `global.css`

The last three columns are not a marketing exaggeration; they are literally the steps, and they are what `CONTENT-GUIDE.md` documents.

Where the name comes from

Astro is an open-source tool, free, widely used, and specifically designed for this kind of site — content-led sites where most of the page is text and pictures rather than an application people log into. It is a good fit for a portfolio.

---

5. Build time, not visit time

This is the one technical idea genuinely worth holding on to, because most of the advantages follow from it.

There are two distinct moments in the life of your website.

Build time — when you run `npm run build` on your own computer. Astro reads your content files, applies the templates, and writes out finished pages into a folder called `dist`. This takes about half a second. It happens on your machine, and no visitor is involved or affected.

Visit time — when a customer opens your site. The host hands them a finished file. Nothing is assembled, nothing is calculated, no database is consulted.

All of the work has been moved out of the moment when a customer is waiting, and into a moment when nobody is waiting.

Compare with WordPress, where every single visit triggers the assembly: PHP starts up, queries a database several times, builds the page, sends it. That work is repeated for every visitor, every time, for a page that has not changed in six months.

This is why:

- **The site is fast.** There is nothing to do but send a file.
- **It cannot be hacked in the usual ways.** There is no database to break into, no login page to attack, no plugin with a vulnerability. There is no software running on your host at all — only files sitting there.
- **It cannot go down under load.** A hundred people arriving at once is a hundred file downloads, which is what web hosts are built for.

- **Hosting is cheap and unfussy.** Any host will do, including the cheapest shared hosting.
- **There is nothing to maintain.** No updates, no security patches, no renewals. A site built today will still work untouched in five years.

The trade-off is real and worth stating plainly: **the site only changes when you rebuild it.** There is no editing a live page from your phone. For a portfolio updated when a project finishes, that is a fair swap. For a news site publishing hourly, it would not be.

---

6. The three layers

The project is deliberately arranged so that three different concerns stay apart from one another. This is the single most important thing to understand about maintaining it.

Layer 1 — Content: what the site says

```
src/content/projects/     one file per project
src/content/pages/        one file per page
src/content/capabilities/ one file per service card
src/site.config.ts        brand, menu, contact details, home page order
```

This is the layer you touch. It is plain text, it has no code in it, and `CONTENT-GUIDE.md` covers it in full.

Layer 2 — Structure: what shape things take

```
src/components/    the header, footer, project card, media player
src/layouts/       the page wrapper every page sits inside
src/pages/         the rules that turn content files into web addresses
```

This is the machinery. You will rarely open it. Changing it means changing what a project page *is*, rather than what it *says* — for example, moving the facts panel from the right-hand side to underneath the description.

Layer 3 — Style: how it looks

```
src/styles/global.css   colours, fonts, sizes, spacing
astro.config.mjs        which fonts get downloaded
```

Covered in section 12.

The value of the separation is that you can work in any one layer without understanding the other two. Adding twenty projects requires no knowledge of how a project page is built. Changing every colour on the site requires no knowledge of what any page says.

---

7. A page being built, step by step

Concretely, here is what happens when you run `npm run build` and Astro reaches your Riverside Tower project.

1. **It reads the content file.** `src/content/projects/riverside-tower-facade.md` gives it the title, the client, the image, and your written description.
2. **It checks the file is valid.** Every project must have a title, a client, a type, a summary and a poster image, and the type must be one of three specific words. If something is missing or misspelt, the build **stops** and names the file and the field.

   This is a feature, not an obstacle. A typo becomes an error message on your own screen, rather than a broken page a customer finds three weeks later.
3. **It picks the template.** The file is a project, so it uses `src/pages/work/[...slug].astro` — the design shared by every project page.
4. **It fills the template in,** dropping your title into the heading, your image into the media frame, your facts into the panel.
5. **It wraps it in the page shell,** adding the header, menu and footer that every page shares.
6. **It works out the neighbours** so the previous/next links at the bottom point at the right projects — recalculated automatically, so adding a project in the middle does not leave stale links.
7. **It writes the finished file** to `dist/work/riverside-tower-facade/index.html`, which is a complete, ordinary web page.

Then it does the same for the other thirteen pages, generates a sitemap for Google, and stops. Total time, about half a second.

Delete that project file and rebuild, and the page is simply not written this time. It vanishes from the Work listing and the home page, and the neighbouring projects re-link around the gap — because none of those things were ever written down by hand.

---

8. What this buys you

Pulling the practical consequences together:

Adding a project is one file. Not five, and not five with four chances to forget one.

Mistakes are caught on your computer, not by your customers. Misspell the project type and the build stops with a message naming the file. There is no equivalent safety net in hand-written HTML, where a typo just produces a slightly wrong page.

The site cannot become internally inconsistent. The menu is generated from the pages that exist. The Work listing is generated from the projects that exist. They cannot disagree with each other, because neither is maintained separately.

It is fast on a building site. Pages are pre-built and small; images load only as they are scrolled towards; videos are not downloaded until needed. That matters when a contractor opens your site on 4G in a portakabin.

It stays fast as it grows. Fifty projects build in about the same time as eight, and each page is still just a file.

You are not locked in. The output is ordinary HTML. If you abandoned Astro tomorrow, the `dist` folder would keep working forever, and your content is plain text files, not rows in somebody's database.

---

9. Why not WordPress, Wix or Squarespace

These are reasonable options, and worth being honest about: their big advantage is editing the site in a browser, from anywhere, with no editor to install. If that mattered most, one of them would be the right answer. You were offered a hosted system at the outset and chose files instead.

What you avoid by not using them:

	YOUR SITE	WORDPRESS
Speed	Sends a finished file	Builds each page per visit
Security	Nothing to attack	Logins, plugins and a database to keep patched
Maintenance	None	Core, theme and plugin updates, indefinitely
Hosting	Any host, cheapest tier	Needs PHP, a database, and enough memory
Monthly cost	Hosting only	Hosting, often plus paid plugins
Longevity	Works untouched for years	Neglect it and it breaks or gets hacked

Wix and Squarespace remove the maintenance burden, but replace it with a subscription and a boundary: you can do what the platform allows, at the speed the platform runs, and exporting your site later ranges from awkward to impossible.

The specific thing they are all weakest at is the thing your site is *for* — serving heavy 3D imagery and video quickly to someone on a phone signal.

---

10. Why not React or Next.js

These are the tools behind large web applications — Facebook, Airbnb, an online banking dashboard. They are built for interfaces where the screen constantly changes in response to the user: filtering, dragging, live updating, logging in.

A portfolio is not that. It is pages of text and pictures that sit still.

Building it in React would mean sending a large amount of JavaScript to every visitor's phone so that their phone could assemble a page that could simply have been sent to them finished. It is slower, heavier and more fragile, for a capability the site never uses.

Astro's specific design decision is that it ships **no JavaScript by default**, and lets you add small pieces only where interactivity is genuinely needed. Your site uses about two kilobytes in total, for exactly three jobs:

- opening and closing the menu on a phone
- playing a video preview when you hover a project card
- filtering the Work page when a filter button is clicked

Everything else is plain HTML and CSS. Turn JavaScript off entirely and the site still works — you would simply see the still images rather than the hover previews.

---

11. Being fair: when plain HTML would have been fine

It would be dishonest to claim this approach always wins.

If the site were three pages that never changed, hand-written HTML would be perfectly sensible, and would need no tooling, no commands and no folder of software. The duplication argument only bites once there is enough repetition for a human to get it wrong.

The break-even is roughly: more than a handful of pages, sharing a common header and footer, that somebody expects to keep adding to. Your site is fourteen pages growing towards fifty, with every project page sharing one design. That is comfortably past the line.

There is a genuine cost, and it is worth naming: **you need Node.js installed and you need to run a command to publish.** That is a real barrier that a hand-written site would not have, and it is why `CONTENT-GUIDE.md` spells out each step. In exchange, adding a project went from five careful edits to one file.

---

12. Changing how the site looks

Everything visual is controlled from one file: **`src/styles/global.css`**.

Open it and the first thing you will see is a block that starts with `:root`. These are the **design tokens** — a set of named values that the rest of the site refers to instead of repeating.

Nothing on the site says “use orange”. Everything says “use the accent colour”, and the accent colour is defined once. Change that one definition and every button, link, bullet and highlight follows, consistently, everywhere.

The idea in one example

Find this line:

```
--accent: #ff6a1a;
```

That is the orange. Change it to, say, a strong blue:

```
--accent: #1a6aff;
```

Save. With the preview running, the whole site rebrands in front of you: buttons, the logo mark, hover states, list bullets, the underline on the current menu item, the glow behind the closing panel. One line.

> `#ff6a1a` is a hex colour code — a standard way of writing a colour that every design tool understands. In Photoshop, Figma or Illustrator, the colour picker shows this code and you can copy it straight across.

The tokens worth knowing

Colours

TOKEN	WHAT IT CONTROLS
`--accent`	The brand colour: buttons, links, highlights
`--accent-hover`	The lighter shade when the mouse is over a button
`--accent-ink`	Text colour *on top of* an accent button — keep it readable
`--bg`	The main page background
`--bg-raised`	Slightly lighter panels, e.g. the “What we make” band
`--bg-sunken`	Slightly darker areas, e.g. the footer
`--surface`	Cards and form fields
`--text`	Main text
`--text-muted`	Secondary text, such as project summaries
`--text-faint`	Small print and labels
`--border`	The faint dividing lines

Text size — `--step--1` up to `--step-5`, smallest to largest. `--step-0` is body text; `--step-5` is the giant home page headline.

These use a fluid scale, which is worth understanding because it is unusual: each size is written as a range rather than a fixed number, and the browser picks a value in between based on the width of the screen. Text therefore grows smoothly from phone to laptop rather than jumping at fixed points. To make all headings bigger, raise the numbers in `--step-3` through `--step-5`.

Spacing — `--space-xs` up to `--space-3xl`, and `--gutter`, which is the margin down the left and right of every page. Increase `--space-3xl` to make the site feel airier; reduce it to tighten everything up.

Shape

TOKEN	WHAT IT CONTROLS
`--radius`	Corner rounding on buttons and inputs
`--radius-lg`	Corner rounding on images, cards and panels
`--container`	Maximum page width on a large screen
`--header-height`	Height of the bar at the top

Set both radius values to `0px` for hard, square, industrial corners.

Changing the fonts

Fonts are set in `astro.config.mjs`. The site currently uses Archivo for headings and Inter for body text, both from Google Fonts.

Find the `name:` line in each block and change it to any font listed at fonts.google.com, spelled exactly as Google spells it:

```
provider: fontProviders.google(),
name: 'Archivo',            <- change this
cssVariable: '--font-display',
weights: [500, 600, 700],
```

Leave `cssVariable` alone — that is the internal name the stylesheet uses.

Do check that the weights you have asked for actually exist for your chosen font; some only offer 400 and 700. The font's page on Google Fonts lists them.

The fonts are downloaded once when the site is built and then served from your own site, so no visitor request ever goes to Google. That is better for privacy and for speed, and it means the site is not broken by Google changing anything.

Changing one specific thing

Tokens handle anything site-wide. To change something in one place only — the header a bit taller, the project cards spaced further apart — the rules for each piece live at the bottom of that piece's own file, inside a `<style>` block.

So `src/components/Header.astro` contains the header's appearance, and `src/components/ProjectCard.astro` the cards'. Those styles apply *only* to that component, so experimenting in one cannot accidentally break another.

A sensible order of attack: try a token first, since it is one line and consistent. Go into a component only when you genuinely want that one thing to differ from everything else.

How the responsive layout is controlled

The rules that rearrange the site for smaller screens are the lines beginning `@media (min-width: ...)`. They read as "once the screen is at least this wide, also apply these rules".

The site is written **mobile first**: the plain rules describe the phone layout, and the `@media` blocks progressively add to it as space becomes available. The three sizes it was tested at are 390px (phone), 834px (tablet) and 1440px (laptop).

The main threshold is `56rem` — about 900px — where the menu changes from a button-and-drawer to inline links across the top.

Three things the tokens do not reach

The accent colour is used in 32 places across the site, and changing that one line genuinely updates all of them — this was tested by switching it to blue and confirming no orange survived anywhere in the built stylesheet.

Three things sit outside the stylesheet, though, and will keep the old colour until they are updated separately. If you rebrand, remember these:

WHAT	WHERE	WHY IT IS SEPARATE
The tab icon	`public/favicon.svg`	An image file, not a stylesheet. Open it in a text editor and change the colour code inside, or replace the file
The phone browser bar tint	The `theme-color` line in `src/components/BaseHead.astro`	Colours the browser chrome on a phone; it has to be a literal value
The placeholder images	`public/media`	The orange is baked into the pictures. Irrelevant once real renders replace them

None of these will break anything if you forget. You will simply have a site in your new colour with an orange tab icon, which is the sort of detail that is easy to miss for months.

Working safely

1. Have the preview running (`npm run dev`) so you can see each change land.
2. Change **one thing**, then look. It is much easier to judge, and much easier to undo.
3. `Command` + `Z` in your editor undoes anything.
4. Check a phone width as well as your laptop. In Chrome, press `F12` and click the small phone icon to simulate one.

Colour, size and spacing changes are safe to experiment with — the worst outcome is that it looks wrong and you undo it. The things to be more careful with are `position` , `display` and `grid` rules inside components, which govern layout rather than appearance.

---

13. The limits of this setup

Stated plainly, so nothing comes as a surprise later:

You cannot edit the site from a phone or a hotel browser. Changes need the project folder and a computer with Node.js installed.

Publishing is a command, not a button. `npm run build` , then upload. It can be automated later if that becomes annoying.

A contact form needs a third party. There is no software running on your host to receive a form submission, so it has to be posted to a service such as Formspree. Until one is configured, the contact page shows your email and phone instead.

Anything genuinely live is out of scope — user accounts, a searchable database of hundreds of products, real-time stock. None of these are on the horizon for a portfolio, and each would mean revisiting the choice.

Someone technical is needed for structural change. Restyling is approachable. Adding a fundamentally new *kind* of page is a developer job — though a small one, because the pieces are already separated.

In one paragraph

Your website is a folder of ordinary files, which is the simplest and most durable thing a website can be. Astro is the tool that writes those files for you from your content, so that you never maintain the same header fourteen times. All the work happens on your computer before anyone visits, which is why the site is fast, cannot be hacked in the usual ways, costs almost nothing to host, and will still work in ten years. The price is that publishing is a command rather than a button — and in exchange, adding a project is one file.